Software Design Document

SENG 271 TERM PROJECT — SDD V2

Project Title	Department Course Timetable Management System
Document Version	v2 (Complete Architecture & Algorithm Specification)
Related SRS Version	SRS v2
Date	10.11.2026
Team Number	Team 19
Team Members	Eren KARAKAŞ (@ErennKarakass) • Semih Mertcan İÇYER (@semihicyer)
Repository URL	https://github.com/ErennKarakass/Daviser.git

Revision History

Version	Date	Author(s)	Summary of Changes
v1	24.10.2026	Team 19	Initial class diagram draft and preliminary scheduling flowchart.
v2	10.11.2026	Team 19	Full object architecture matching Code v1 Java skeleton; added all 13 system classes, detailed 11-step algorithm flowchart, pseudocode (Section 6.3), hard/soft constraint matrices, and full requirements traceability.
Final	[Pending]	Team 19	Final system design synchronized with completed Checkpoint 3 release.

1. Introduction

This Software Design Document (SDD) specifies the complete software architecture, class models, data structures, and automatic timetable generation algorithm for the Department Course Timetable Management System. Based on SRS v2, this document provides the architectural specification for the Java codebase delivered across project checkpoints.

2. System Overview

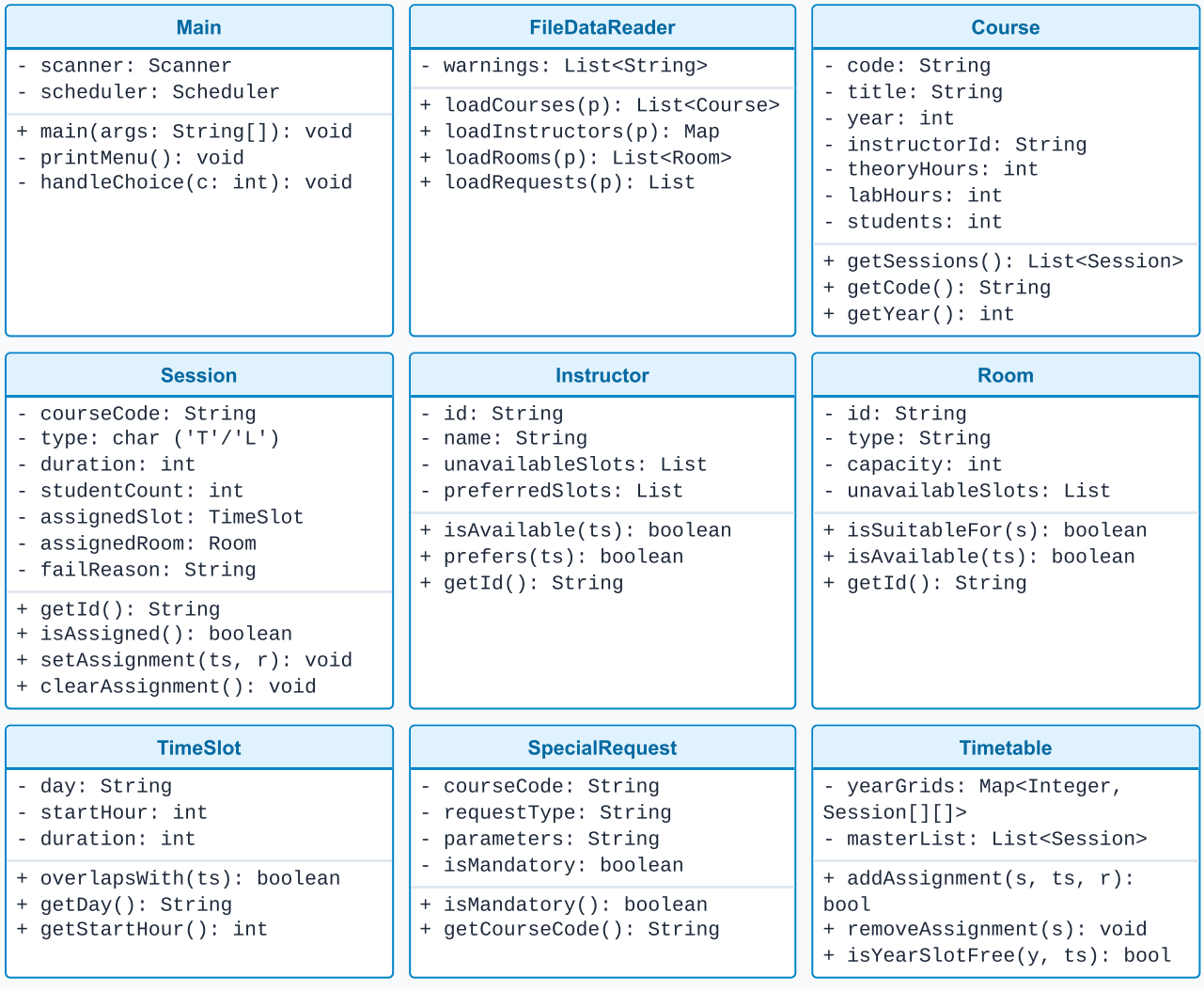
System Subsystem	Participating Java Classes	Subsystem Responsibilities
User Interface	`Main`	Renders interactive console screens (Screens 1–7), captures user input, displays grids, and handles menu flow.
Input Processing	`FileDataReader`	Reads CSV/TXT data files, parses numerical values, validates entity references, and logs line-by-line warnings.
Domain Model	`Course`, `Session`, `Instructor`, `Room`, `TimeSlot`, `SpecialRequest`	Encapsulates academic courses, teaching blocks (-T/-L), instructor availability, room capacities, and request rules.

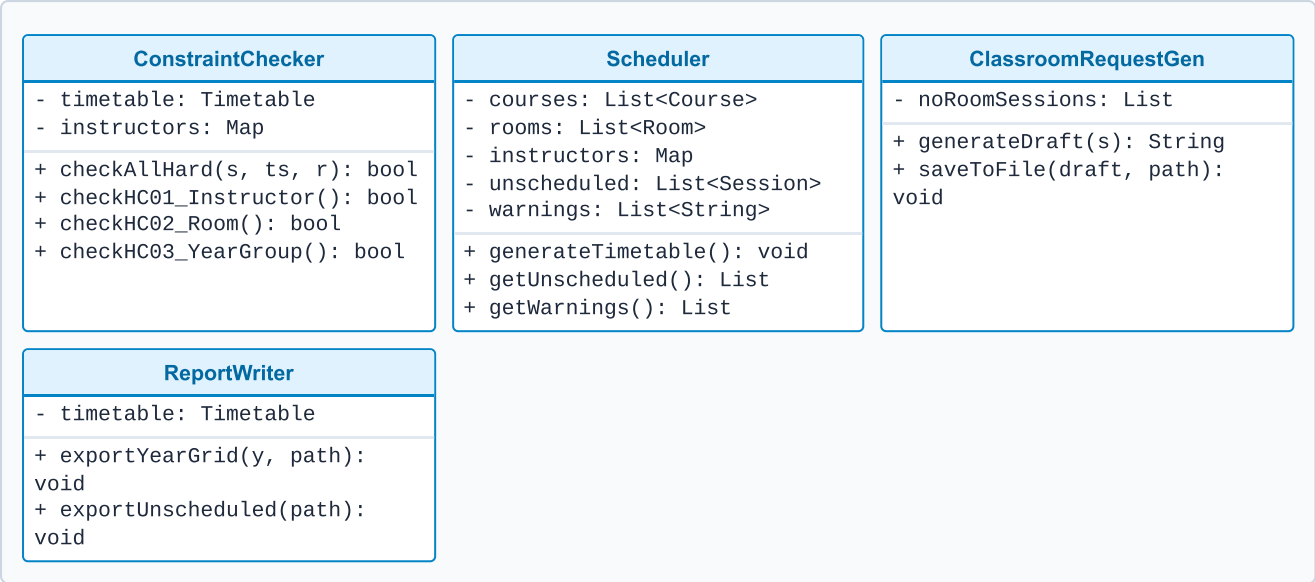
System Subsystem	Participating Java Classes	Subsystem Responsibilities
Scheduling Engine	`Scheduler`, `ConstraintChecker`, `Timetable`	Executes rule-based automatic assignment loops, evaluates hard/soft constraints (HC-01..09, SC-01..04), and stores placements.
Output & Reporting	`ClassroomRequestGenerator`, `ReportWriter`	Formats year-group timetables, isolates NO ROOM failures, drafts classroom request emails, and exports output files.

3. UML Class Diagram

The class architecture of the application is presented below. All 13 classes, attribute visibilities (- private, + public), and method signatures correspond strictly to the Java source files in `src/`.

Complete UML Class Architecture (13 Classes — Code v1 Match)





4. Class Descriptions

Class Name	Primary Responsibility (One Sentence)	Key Attributes	Key Methods
`Main`	Entry point managing console interaction, displaying menu options, and orchestrating user choices.	`scanner, scheduler`	`main(), printMenu(), handleChoice()`
`FileDataReader`	Reads external CSV and TXT files, validates records, creates domain objects, and logs parse warnings.	`warnings`	`loadCourses(), loadInstructors()`
`Course`	Stores course curriculum details and splits theoretical and lab hours into schedulable Session blocks.	`code, title, year, theoryHours, labHours, students`	`getSessions(), getCode()`
`Session`	Represents an individual teaching block (-T or -L) with duration, student count, and assignment state.	`courseCode, type, duration, studentCount, assignedSlot, assignedRoom`	`isAssigned(), setAssignment(), clearAssignment()`
`Instructor`	Stores instructor profiles, assigned courses, unavailable time slots, and preferred teaching hours.	`id, name, unavailableSlots, preferredSlots`	`isAvailable(), prefers()`
`Room`	Holds room capacity, room type (Classroom/Lab), and unavailable maintenance time slots.	`id, type, capacity, unavailableSlots`	`isSuitableFor(), isAvailable()`
`TimeSlot`	Encapsulates a day and start hour interval representing a discrete 1-hour scheduling slot.	`day, startHour, duration`	`overlapsWith(), getDay()`
`SpecialRequest`	Stores special course constraints (fixed time, designated room) and marks whether mandatory.	`courseCode, requestType, parameters, isMandatory`	`isMandatory(), getCourseCode()`

Class Name	Primary Responsibility (One Sentence)	Key Attributes	Key Methods
`Timetable`	Maintains year-group 2D assignment grids and provides fast slot occupation lookups.	`yearGrids`, `masterList`	`addAssignment()`, `removeAssignment()`, `isYearSlotFree()`
`ConstraintChecker`	Evaluates all mandatory hard constraints (HC-01..09) and soft preference warnings for proposed slots.	`timetable`, `instructors`	`checkAllHard()`, `checkHC01_Instructor()`
`Scheduler`	Implements the core rule-based automatic scheduling algorithm, managing iteration loops and failures.	`courses, rooms, instructors, unscheduled, warnings`	`generateTimetable()`, `getUnscheduled()`
`ClassroomRequestGen`	Constructs formal email drafts for unscheduled sessions failing due to room shortages (NO ROOM).	`noRoomSessions`	`generateDraft()`, `saveToFile()`
`ReportWriter`	Exports formatted weekly timetables, unscheduled reports, and summary files to the `output/` directory.	`timetable`	`exportYearGrid()`, `exportUnscheduled()`

5. Design Decisions

Architectural Question	Selected Design Choice	Rationale & Impact Analysis
1. Session Splitting	Decompose `Course` into independent `Session` objects for theory (-T) and lab (-L).	Allows theoretical and lab blocks to be placed independently in different rooms (Classroom vs Lab) and time slots while keeping track of their parent course.
2. TimeSlot Model	Immutable `TimeSlot` object holding `day` (Mon-Fri) and `startHour` (9 to 16).	Ensures thread-safe slot comparisons and overlap calculations without risks of side-effect mutations during trial loops.
3. Timetable Storage	Dual structure: `Session[5][8]` 2D array per year group + `ArrayList<Session>` master list.	The 2D array yields instant O(1) grid rendering for Screen 4; the master list enables efficient iteration during room and instructor availability checks.
4. Availability Model	`List<TimeSlot>` attributes stored inside `Instructor` and `Room` entities.	Enables direct checking of HC-07 (Instructor Availability) and HC-09 (Room Availability) without complex external lookup tables.
5. Special Requests	Encapsulated in a `SpecialRequest` class with an explicit `isMandatory` flag.	Fixed-time mandatory requests are scheduled first in Step 1; preferred requests are processed as soft warnings if violated.
6. Constraint Architecture	Centralized `ConstraintChecker` class with explicit boolean methods per HC code.	Isolates rule validation logic from scheduling loops, facilitating clean code reuse during manual schedule modifications (Screen 6).

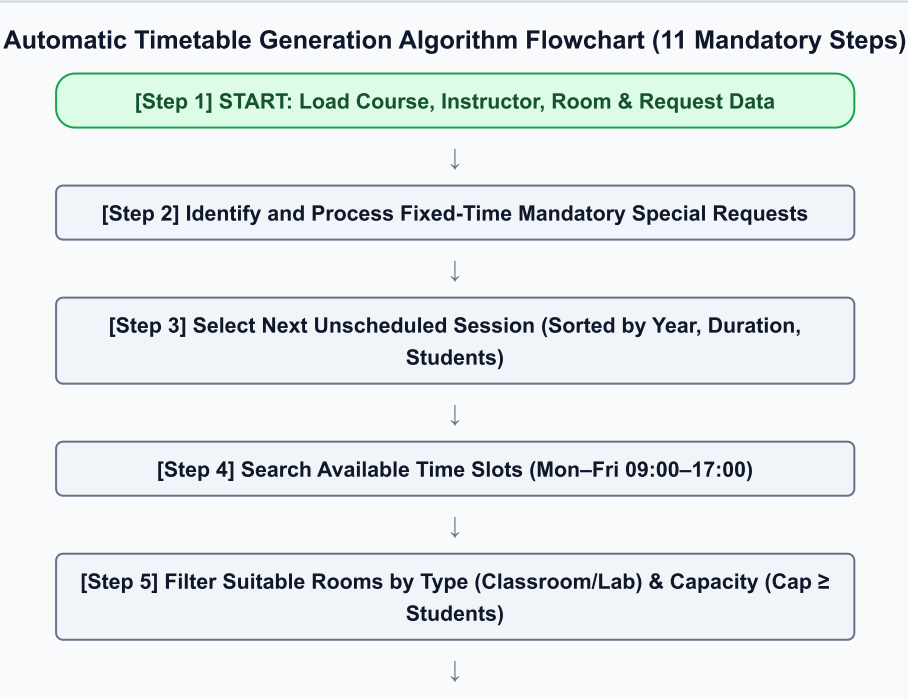
Architectural Question	Selected Design Choice	Rationale & Impact Analysis
7. Input Error Handling	Try-catch blocks in `FileDataReader` that log line numbers and skip malformed lines.	Guarantees NFR-01 (Reliability) by preventing invalid data rows (e.g., string in numeric field) from crashing the program.
8. Java Collections Selection	`ArrayList` for ordered sessions/rooms; `HashMap<String, Course>` for ID lookups.	Optimizes lookup performance during data loading while maintaining predictable iteration order during algorithm execution.

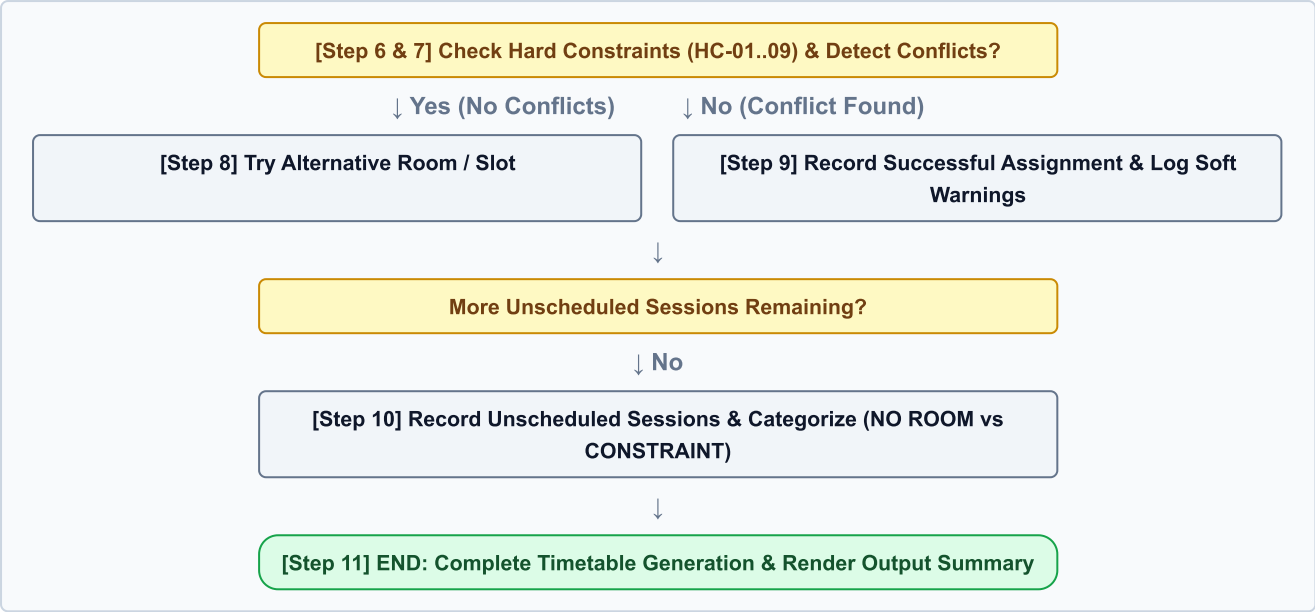
6. Automatic Scheduling Algorithm

6.1 Scheduling Strategy (Oral Defense Answers)

1. **Session Order:** Fixed-time mandatory requests are scheduled first. Remaining sessions are sorted by year group, then duration (descending), then student count (descending).
2. **Time Slot Selection:** Candidate time slots are scanned sequentially from Monday 09:00 to Friday 16:00. Morning slots (09:00–12:00) are evaluated first to favor instructor preferences (SC-02).
3. **Room Selection:** Rooms matching the required type (Classroom/Lab) are sorted by capacity ascending (best-fit strategy to reserve large rooms for larger classes).
4. **Constraint Verification:** Each candidate (Session, TimeSlot, Room) tuple is evaluated sequentially against HC-01 through HC-09. The first failing rule aborts candidate evaluation.
5. **Conflict Resolution:** Upon conflict, the algorithm tests the next candidate room; if all rooms fail, it advances to the next candidate time slot.
6. **Unscheduled Determination:** If all 40 slots and suitable rooms fail, the session is moved to `unscheduled` and tagged as NO ROOM (capacity/type failure) or CONSTRAINT.

6.2 Algorithm Flowchart





6.3 Algorithm Pseudocode

```
FUNCTION generateTimetable(): // Step 1: Load and parse input data courses, instructors, rooms, requests = loadAllData() // Step 2: Process fixed-time mandatory requests FOR EACH req IN requests WHERE req.isMandatory AND req.type == "FIXED_TIME": session = findSession(req.courseCode) targetSlot = req.targetSlot targetRoom = findSuitableRoom(session, targetSlot) IF checkAllHardConstraints(session, targetSlot, targetRoom) THEN: assign(session, targetSlot, targetRoom) ELSE: recordFailure(session, "Fixed time request conflicts with hard constraints") // Step 3: Sort remaining unscheduled sessions remainingSessions = getUnscheduledSessions() sortDescendingByYearAndDuration(remainingSessions) // Steps 4 to 9: Main scheduling loops FOR EACH session IN remainingSessions: placed = FALSE FOR EACH slot IN getWorkingTimeSlots(): // Mon-Fri, 9:00-17:00 IF placed THEN BREAK FOR EACH room IN getSortedSuitableRooms(session): // Capacity >= Students IF checkAllHardConstraints(session, slot, room) THEN: assign(session, slot, room) checkAndLogSoftConstraints(session, slot, room) placed = TRUE BREAK // Break room loop // Step 10: Record unscheduled sessions IF NOT placed THEN: IF noRoomFitsCapacityOrType(session) THEN: markUnscheduled(session, "NO ROOM", "No laboratory/classroom with required capacity") ELSE: markUnscheduled(session, "CONSTRAINT", "Slot/Instructor constraint overlap") // Step 11: Render generation summary printSummaryReport()
```

6.4 Hard Constraint Verification Matrix

ID	Hard Constraint	Implementing Java Method	Target Code Version
HC-01	Instructor Conflict	`ConstraintChecker.checkHC01_Instructor(session, slot)`	Code v3
HC-02	Room Conflict	`ConstraintChecker.checkHC02_Room(room, slot)`	Code v3
HC-03	Year-Group Conflict	`ConstraintChecker.checkHC03_YearGroup(session, slot)`	Code v3
HC-04	Working Hours	`TimeSlot.isValidWorkingHour(slot)`	Code v1
HC-05	Room Capacity	`Room.isSuitableFor(session)`	Code v2
HC-06	Room Type	`Room.isSuitableFor(session)`	Code v2

ID	Hard Constraint	Implementing Java Method	Target Code Version
HC-07	Instructor Availability	`Instructor.isAvailable(slot)`	Code v2
HC-08	Fixed-Time Requirements	`Scheduler.processFixedRequests()`	Code v3
HC-09	Room Availability	`Room.isAvailable(slot)`	Code v2

6.5 Known Limitations

Because the scheduling algorithm employs a rule-based greedy heuristic without deep backtracking, the outcome depends on the order in which sessions are evaluated. In rare edge cases with tight room constraints, a session may remain unscheduled even if a valid schedule theoretically exists. This deliberate trade-off keeps algorithmic complexity manageable and avoids NP-hard optimization libraries as specified in project guidelines.

7. Conflict Detection and Handling

Conflict Condition	Detection Method	Algorithmic Action	User Console Output
Instructor / Room Overlap	`checkHC01` / `checkHC02`	Reject candidate room/slot; test next available room.	`[FAIL] HC-01/02: Time slot busy`
Year-Group Overlap	`checkHC03_YearGroup`	Skip slot for that year group; advance to next slot.	`[FAIL] HC-03: Year group collision`
Room Capacity Deficit	`Room.isSuitableFor`	Mark session unscheduled; categorize as NO ROOM.	`[FAIL] HC-05: No room cap >= X`
Fixed-Time Conflict	`processFixedRequests`	Reject placement; report constraint violation.	`[FAIL] HC-07: Fixed time unavailable`
Soft Preference Violation	`checkSoftConstraints`	Accept assignment; append warning to summary.	`[WARN] SC-02: Afternoon placement`

8. User Interface Design

Menu Option	Functional Purpose	SRS Requirements	Executed Java Method
1. Load Data	Reads CSV/TXT files and logs validation warnings.	FR-01 to FR-05	`FileDataReader.loadAll()`
2. Generate	Runs automatic timetable generation algorithm.	FR-06 to FR-10	`Scheduler.generateTimetable()`
3. View Year	Renders 5x8 weekly grid for selected year group.	FR-11	`Main.renderYearTimetable()`
4. View All	Lists all scheduled session assignments.	FR-12	`Main.renderAllScheduled()`
5. View Unscheduled	Displays unscheduled sessions and soft warnings.	FR-13, FR-14	`Main.renderUnscheduled()`
6. Modify / Remove	Manually moves or clears a session assignment.	FR-15 to FR-17	`Timetable.modifyAssignment()`
7. Email Draft	Drafts classroom request text for NO ROOM sessions.	FR-18 to FR-20	`ClassroomRequestGen.generate()`

Menu Option	Functional Purpose	SRS Requirements	Executed Java Method
8. Save Output	Exports timetables and reports to `output/`.	FR-21	`ReportWriter.saveAll()`
0. Exit	Terminates application execution.	NFR-02	`Main.main() exit`

9. Requirements Traceability Matrix

Requirement	Implementing Java Method	Test Case ID	Status
FR-01 to FR-05	`FileDataReader.loadAll()`	TC-12	Implemented
FR-06 to FR-10	`Scheduler.generateTimetable()`	TC-01, TC-09, TC-10	Planned (v3)
FR-11 to FR-14	`Main.renderYearTimetable()`	TC-01	Implemented (Skeleton)
FR-15 to FR-17	`Timetable.modifyAssignment()`	TC-13	Planned (v5)
FR-18 to FR-20	`ClassroomRequestGenerator.generate()`	TC-11	Planned (v6)
HC-01 to HC-09	`ConstraintChecker.checkAllHard()`	TC-02 to TC-07	Planned (v3)

10. Changes from Previous Version (v1 to v2)

Element	Change Type	Reason for Revision
`ConstraintChecker`	Added Class	Extracted hard-constraint validation methods into a dedicated class for clean code architecture.
`ClassroomRequestGen`	Added Class	Added to handle FR-19 and FR-20 room request email generation logic cleanly.
Section 6.3 Pseudocode	Added	Added complete pseudocode for automatic timetable generation to satisfy Checkpoint 1 requirements.
UML Diagram Grid	Expanded	Expanded diagram representation to include all 13 system classes present in Code v1.

AI-Use Declaration

Tool	Used For	Verification Method
ChatGPT / Claude / Gemini	Assisting in pseudocode formatting, UML class grid structuring, and table alignment.	Both team members verified that all class names and signatures match Java source files in `src/` exactly.